

Chapter 4: Informed Search Algorithms (Exam-Focused Notes)

0. Notation and What We Measure

Symbols

- b : maximum branching factor.
- d : depth of the shallowest goal (unit-cost setting).
- m : maximum depth of the search space (may be infinite).
- $g(n)$: path cost from start to node n .
- $h(n)$: heuristic estimate of cheapest cost from n to a goal.
- $h^*(n)$: true cheapest cost from n to a goal.
- $f(n)$: evaluation function used to rank nodes for expansion.
- C^* : optimal solution cost (minimum cost of any goal path).
- ε : minimum step cost, i.e., every step cost satisfies $c(x, a, n') \geq \varepsilon$ and $\varepsilon > 0$.

Time vs Space Complexity (Exam Standard)

- **Time complexity**: number of nodes generated and/or expanded in the worst case.
- **Space complexity**: maximum number of nodes stored at once (frontier, plus closed set if used).

Geometric Series Toolkit

Worst case, nodes at depth i are at most b^i . Total nodes up to depth k is

$$\sum_{i=0}^k b^i = 1 + b + b^2 + \dots + b^k = \frac{b^{k+1} - 1}{b - 1} \quad (b > 1).$$

The largest term dominates, so

$$1 + b + \dots + b^k = O(b^k).$$

1. Review: Generic Tree Search

Listing 1: Pseudocode: TREE-SEARCH

```
function TREE-SEARCH(problem, fringe) returns solution or failure
  fringe <- INSERT(MAKE-NODE(INITIAL-STATE(problem)), fringe)
  loop:
    if fringe is empty then return failure
    node <- REMOVE-FRONT(fringe)
    if GOAL-TEST(STATE(node)) then return SOLUTION(node)
    fringe <- INSERTALL(EXPAND(node, problem), fringe)
```

A search **strategy** is defined by how the frontier (fringe) is ordered, i.e., how REMOVE-FRONT selects the next node.

2. Best-First Search (General Framework)

Core Idea

Assign each node an evaluation score (“desirability”), then expand the node with the best score next.

Implementation

- Frontier is a **priority queue** ordered by an evaluation function $f(n)$.
- Different choices of $f(n)$ produce different algorithms.

Special Cases

- Greedy Best-First Search: $f(n) = h(n)$.
- A* Search: $f(n) = g(n) + h(n)$.

3. Greedy Best-First Search

Strategy

Greedy best-first search always expands the node that *looks* closest to the goal according to the heuristic.

$$f(n) = h(n).$$

Intuition: “Go to the state that appears nearest to the goal.”

Dry Run (Priority Queue Trace)

Assume we store the frontier as a min-priority queue by h . Example values:

$$h(\text{Arad}) = 366, \quad h(\text{Sibiu}) = 253, \quad h(\text{Timisoara}) = 329, \quad h(\text{Zerind}) = 374.$$

Step	Frontier before pop (lowest h first)	Expanded	Frontier after insert
0	[(Arad,366)]	–	[(Arad,366)]
1	[(Arad,366)]	Arad	[(Sibiu,253),(Timisoara,329),(Zerind,374)]
2	[(Sibiu,253),(Timisoara,329),(Zerind,374)]	Sibiu	[... (children of Sibiu inserted and sorted by h) ...]

Completeness

- **Not complete** in general: it can get stuck in loops (example pattern $A \rightarrow B \rightarrow A \rightarrow B \rightarrow \dots$).
- In a **finite** state space, greedy best-first can be made complete by adding repeated-state checking (a closed set).

Optimality: Why Greedy Best-First is NOT Optimal (Must Know)

Greedy best-first search is not optimal because it **ignores the cost already paid** to reach a node.

Key reason. It chooses the smallest $h(n)$, but the true solution cost through n is

$$\text{true cost through } n = g(n) + h^*(n).$$

Greedy uses only $h(n)$ and completely discards $g(n)$. Therefore it can prefer a node that “seems close” to the goal even if reaching that node was extremely expensive.

Concrete counterexample (graph with costs). Let start be S , goal be G .

- Path 1 (optimal): $S \rightarrow A \rightarrow G$ with costs $c(S, A) = 1$, $c(A, G) = 1$. Total cost = 2.
- Path 2 (suboptimal): $S \rightarrow B \rightarrow G$ with costs $c(S, B) = 100$, $c(B, G) = 1$. Total cost = 101.

Define a heuristic that makes B look closer:

$$h(A) = 10, \quad h(B) = 0, \quad h(G) = 0.$$

Greedy compares $h(A) = 10$ and $h(B) = 0$, so it expands toward B first and can quickly reach G via the expensive route, returning cost 101, even though a cost-2 solution exists.

Summary. Greedy best-first can return the first goal found, but “first found” does not imply “lowest cost” because the algorithm is not using $g(n)$ at all.

Time Complexity: $O(b^m)$ (Full Derivation)

Worst case, a heuristic gives no useful guidance and greedy explores deep into the search space.

1. In the worst case, greedy best-first expands nodes down to maximum depth m before finding a goal (or may fail to terminate in infinite spaces).
2. Nodes up to depth m in a b -ary tree are

$$\sum_{i=0}^m b^i = O(b^m).$$

$$\boxed{T_{\text{Greedy}} = O(b^m)}.$$

Space Complexity: $O(b^m)$ (Full Derivation)

Greedy best-first maintains a priority queue frontier containing many generated nodes that are not yet expanded.

1. In the worst case, it can generate essentially all nodes up to depth m before terminating.
2. Those nodes remain in memory either in the frontier or as bookkeeping.

$$S_{\text{Greedy}} = O(b^m).$$

Watch Out

- “Good heuristic” does not imply optimal for greedy.
- Greedy may perform well in practice but has no optimality guarantee.
- Without repeated-state checking, greedy can loop forever in cyclic graphs.

Visual Sketching Guide

Draw a tree and show greedy “jumping” toward the node with smallest h even if that path is long. Label nodes by h and show the algorithm chasing the smallest label.

Self-Check

1. In one sentence: why does ignoring $g(n)$ make greedy non-optimal?
2. Give a small example where a node has smaller h but much larger g than an alternative.
3. Why can greedy loop in graphs without repeated-state checking?

4. A* Search

Strategy

A* combines the cost so far and the estimated remaining cost:

$$f(n) = g(n) + h(n).$$

Interpretation: $f(n)$ is an estimate of the total cost of a solution path that goes through n .

Dry Run (Priority Queue Trace with $f = g + h$)

Example values at start node Arad:

$$g(\text{Arad}) = 0, h(\text{Arad}) = 366, f(\text{Arad}) = 366.$$

Suppose Arad has successors:

Sibiu: $g = 140, h = 253, f = 393$; Timisoara: $g = 118, h = 329, f = 447$; Zerind: $g = 75, h = 374, f = 449$.

Step	Frontier before pop (lowest f first)	Expanded	Frontier after insert
0	[(Arad,366)]	–	[(Arad,366)]
1	[(Arad,366)]	Arad	[(Sibiu,393),(Timisoara,447),(Zerind,449)]
2	[(Sibiu,393),(Timisoara,447),(Zerind,449)]	Sibiu	[... (children inserted and sorted by f) ...]

Admissibility (“Never Overestimate”)

A heuristic $h(n)$ is **admissible** if it never overestimates the true cheapest cost-to-go:

$$0 \leq h(n) \leq h^*(n) \text{ for all } n.$$

This means h is **optimistic**.

Why Admissibility Makes A* Optimal (Tree Search)

Let C^* be the optimal solution cost. Consider any node n on an optimal solution path. The true cheapest completion from n costs $h^*(n)$, so the true optimal solution through n costs $g(n) + h^*(n) = C^*$ (for nodes on an optimal path, completion cost fits within the optimal total).

If h is admissible, then

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*.$$

So nodes on an optimal path never have $f(n)$ greater than C^* .

A* always expands the node with smallest f . Therefore, A* cannot permanently ignore the optimal path in favor of a strictly more expensive goal path, because the frontier will always contain nodes with $f \leq C^*$ on the optimal route.

What If h is NOT Admissible (Overestimates)?

If $h(n) > h^*(n)$ for some node n , then $f(n) = g(n) + h(n)$ can be **larger than the true best solution cost through n** . This breaks the “optimistic bound” property and causes problems:

- **A* can lose optimality.** Nodes on the optimal path can get inflated f values, so A* may expand toward and reach a suboptimal goal first.
- **A* can terminate with a suboptimal goal.** Many A* formulations stop when a goal is popped from the priority queue. If the optimal path nodes are artificially delayed by overestimation, a more expensive goal may be popped first.
- **The optimality proof fails.** The key inequality $f(n) \leq C^*$ for nodes on an optimal path is no longer guaranteed, so we cannot conclude A* will expand the optimal route before any suboptimal goal.

Why admissibility is preferable. Admissibility makes $f(n)$ behave like a **lower bound** on the cost of any solution through n . A* uses this property to ensure “first goal popped has cost C^* ” (optimal).

Consistency (“Triangle Inequality” / Monotonicity)

A heuristic h is **consistent** if for every edge $n \rightarrow n'$ generated by action a :

$$h(n) \leq c(n, a, n') + h(n').$$

Rearrange:

$$h(n) - h(n') \leq c(n, a, n').$$

Meaning: your heuristic estimate cannot drop by more than the step cost you paid.

Consistency Implies Non-Decreasing f Along Paths

Let $f(n) = g(n) + h(n)$. For successor n' :

$$f(n') = g(n') + h(n') = g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n).$$

So with consistency:

$$f(n') \geq f(n) \text{ along any path.}$$

Admissibility vs Consistency (Comparison)

Feature	Admissibility	Consistency
What it checks	Node-to-goal estimate	Edge-by-edge (triangle inequality)
Condition	$0 \leq h(n) \leq h^*(n)$	$h(n) \leq c(n, a, n') + h(n')$
Strength	Weaker	Stronger
Relationship	—	Consistency implies admissibility
Why it matters	A* tree-search optimality	A* graph-search optimality without re-opening
Effect on f	No guaranteed monotonicity	Ensures f is non-decreasing along paths

Why Consistency is Needed for Graph Search

In graph search, we typically maintain a closed set of visited states. If h is admissible but **inconsistent**, A* might first reach a state with a suboptimal g and close it, then later discover a cheaper path to the same state. If the implementation never re-opens closed states, optimality can be lost. Consistency prevents this problem because the first time a node is selected for expansion, it is guaranteed to be on a cheapest path to that state.

Completeness and Optimality (A*)

- **Optimal:** With admissible h in tree search; with consistent h in graph search (standard implementation without re-opening).
- **Complete:** Yes under common conditions (e.g., positive step costs and not having infinitely many nodes with $f \leq C^*$).

Time Complexity (Worst Case) and Why It Is Exponential

A* expands all nodes with $f(n) < C^*$ and possibly some nodes with $f(n) = C^*$, and it expands no nodes with $f(n) > C^*$.

The number of nodes satisfying $f(n) \leq C^*$ can still be exponential. A clean worst-case bound is obtained by observing that A* can behave like uniform-cost search when $h(n) = 0$.

Time Complexity: $O(b^{C^*/\epsilon})$ (Full Derivation via Reduction to UCS)

1. Consider the special case $h(n) = 0$ for all n (this is admissible).
2. Then $f(n) = g(n)$ and A* becomes uniform-cost search (UCS).
3. UCS expands all nodes with $g(n) \leq C^*$.

4. If each step cost is at least $\varepsilon > 0$, then any path of k steps costs at least $k\varepsilon$:

$$g(n) \geq k\varepsilon.$$

5. Therefore, any node with $g(n) \leq C^*$ must satisfy

$$k\varepsilon \leq C^* \Rightarrow k \leq \frac{C^*}{\varepsilon}.$$

6. So, in the worst case, A^* may need to generate nodes up to depth

$$k_{\max} = \left\lfloor \frac{C^*}{\varepsilon} \right\rfloor.$$

7. The number of nodes up to depth $k_{\max}$ is

$$\sum_{i=0}^{k_{\max}} b^i = O(b^{k_{\max}}) = O(b^{C^*/\varepsilon}).$$

$$T_{A^* \text{ (worst)}} = O(b^{C^*/\varepsilon}).$$

This shows A^* can be exponential in the worst case.

Space Complexity: $O(b^{C^*/\varepsilon})$ (Full Derivation)

A^* must keep the frontier (priority queue) and, in graph search, the closed set.

1. In the worst case, the number of generated nodes with $f(n) \leq C^*$ is $O(b^{C^*/\varepsilon})$.
2. These nodes are stored either in the frontier or in closed.

$$S_{A^* \text{ (worst)}} = O(b^{C^*/\varepsilon}).$$

Watch Out

- If h overestimates, A^* may return a suboptimal solution.
- A^* is often limited by **memory**, because it keeps many nodes in the frontier.
- For graph search without re-opening closed nodes, you want **consistency**.

Visual Sketching Guide

Draw “ f -contours”: regions of nodes with equal (or similar) f . A^* expands all nodes in lower- f contours before moving to higher- f contours.

Self-Check

1. If $h(n) = 0$ for all n , what does A^* become and why?
2. Write the inequality that shows admissibility implies $f(n) \leq C^*$ for nodes on an optimal path.
3. Why does inconsistency cause trouble with a closed set in graph search?

5. Admissible Heuristics: Examples (8-Puzzle)

Two Standard Heuristics

- $h_1(n)$: number of misplaced tiles.
- $h_2(n)$: sum of Manhattan distances of each tile from its goal position.

Both are admissible because they are based on relaxations:

- For h_1 : relax the rules so a tile can move directly to its goal position in one move (ignoring blocking). Then each misplaced tile needs at least one move.
- For h_2 : relax the rules so a tile can move one step in the grid without being blocked by other tiles. Each tile must move at least its Manhattan distance.

Example Values

For a given start state S , an example calculation yields:

$$h_1(S) = 8, \quad h_2(S) = 18.$$

(Manhattan distance is computed by summing, for each tile, the horizontal plus vertical offset from its goal cell.)

Dominance

If two heuristics are admissible and

$$h_2(n) \geq h_1(n) \quad \text{for all } n,$$

then h_2 **dominates** h_1 and typically leads to fewer node expansions.

If h_a and h_b are admissible, then

$$h(n) = \max(h_a(n), h_b(n))$$

is also admissible and dominates both.

Self-Check

1. Why is h_2 usually larger than (or equal to) h_1 ?
2. Why does taking max preserve admissibility?
3. If two admissible heuristics are available, why is the dominated one usually less useful?

6. Relaxed Problems (How to Design Admissible Heuristics)

Core Method

- Create a **relaxed problem** by removing constraints so the problem becomes easier.
- Let the optimal cost of the relaxed problem be $h_{\text{relax}}(n)$.

- Because the relaxed problem is easier, its optimal cost is a **lower bound** on the real optimal cost:

$$h_{\text{relax}}(n) \leq h^*(n).$$

Therefore h_{relax} is admissible.

Example: TSP Lower Bound Idea

A minimum spanning tree (MST) cost can be computed efficiently and can serve as a lower bound for certain tour variants. The key exam point is the lower-bound logic: relax constraints, solve the easier problem exactly, and use that optimal cost as an admissible heuristic.

Self-Check

1. Why does relaxing constraints never increase the optimal solution cost?
2. What is the single sentence that justifies admissibility from a relaxed problem?
3. Give one relaxation for a problem you know and describe the heuristic it suggests.

7. Decision Tables

Greedy vs A* (High-Yield Comparison)

Algorithm	Complete	Optimal	Time (worst)	Space (worst)
Greedy ($f = h$)	No (general)	No	$O(b^m)$	$O(b^m)$
A* ($f = g + h$)	Yes (common conditions)	Yes (with conditions)	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$

Conditions (State Clearly in Exams)

- Greedy is not optimal because it ignores $g(n)$.
- A* optimality requires admissibility (tree search) and typically consistency (graph search without re-opening).
- Completeness typically requires positive step costs and not having infinitely many nodes with $f \leq C^*$.

8. Watch Out (Common Exam Pitfalls)

- Confusing admissibility with consistency.
- Claiming greedy is optimal if the heuristic is “good”. Greedy has no optimality guarantee.
- Forgetting to explain what fails when h overestimates (A* can return suboptimal solutions).
- Writing “A* is polynomial” – worst-case time and space are exponential.

9. Final Self-Check (Mixed)

1. State admissibility and consistency as formulas.
2. Explain in one paragraph why greedy best-first is not optimal.
3. Explain in one paragraph what can go wrong if h is not admissible in A^* .
4. Prove (in 3 lines) that consistency implies f is non-decreasing along a path.
5. If h_2 dominates h_1 , why does h_2 typically expand fewer nodes?

10. Complexity Cheat Sheet

Algorithm	Time (worst)	Space (worst)	Core Idea
Greedy Best-First	$O(b^m)$	$O(b^m)$	Expand lowest $h(n)$
A^* Search	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$	Expand lowest $g(n) + h(n)$